

Performance Engineering with Python Class Notes

1. What Is Performance Engineering?

Performance engineering is the systematic process of measuring, analyzing, explaining, and improving program performance. In parallel programming, the goal is not only to make programs faster, but to understand why they behave the way they do.

2. Performance Metrics

Key metrics include runtime, speedup, efficiency, and scalability. Speedup measures improvement relative to serial execution, while efficiency measures how well computational resources are used.

Speedup Formula

$S(p) = T_1 / T_p$, where T_1 is serial runtime and T_p is runtime using p cores.

Efficiency Formula

$E(p) = S(p) / p$. Efficiency decreases as overhead and contention increase.

3. Measuring Performance Correctly in Python

Use high-resolution timers and multiple trials. Avoid single measurements.

Timing with `time.perf_counter()`

```
import time

start = time.perf_counter()

# work here

end = time.perf_counter()

print(end - start)
```

Multiple Runs (Median-of-Three)

```
times = []

for _ in range(3):
    start = time.perf_counter()
    work()
```

```
times.append(time.perf_counter() - start)
print(sorted(times)[1])
```

4. Parallel Overhead in Python

Parallel programs incur overhead from process creation, task scheduling, communication, and synchronization. When tasks are too small, overhead can dominate computation.

BAD vs GOOD Parallelism

```
# BAD: fine-grained tasks
with mp.Pool(4) as p:
    p.map(work, data, chunksize=1)

# GOOD: coarse-grained tasks
with mp.Pool(4) as p:
    p.map(work, data, chunksize=10000)
```

5. Compute-Bound vs Memory-Bound

Compute-bound programs are limited by arithmetic operations, while memory-bound programs are limited by memory bandwidth. Arithmetic intensity determines which dominates.

```
# Memory-bound
np.sum(a)

# Compute-bound
for _ in range(20):
    a = a * 1.000001 + 0.000001
```

6. Performance Visualization

Plots reveal performance trends that raw numbers hide. Runtime, speedup, and efficiency curves expose overhead and saturation.

7. NUMA Effects

On NUMA systems, memory access time depends on which socket owns the memory. First-touch memory placement strongly affects parallel performance.

```
# BAD: memory allocated in main process
```

```
a = np.ones(N)

# GOOD: memory allocated inside workers
a = np.ones(chunk_size)
```

Final Takeaways

Performance engineering requires careful measurement, analysis, and explanation.
Understanding overhead, memory behavior, and scalability is more important than achieving raw speed.